

Script Inventory for Cloud Owner & Cloud Admin: LLDAP + OpenFGA + libcloud REST + Vault

Overview

Every script below is a pre-built runbook that the Cloud Owner or Cloud Admin can invoke directly without developer involvement. Scripts are grouped by system layer and persona. Each entry lists the script name, the persona who runs it, the systems it touches, and its exact purpose.

Naming convention used: ``ayer>-<action>-<target>.sh`` (or ``.py`` where API-heavy logic

Group 1 — Identity Management (LLDAP)

These scripts drive all user and group lifecycle operations against LLDAP's GraphQL API.

Script Name	Persona	Purpose
<code>lldap-user-onboard.sh</code>	Cloud Admin	Creates a new LLDAP user account (username, display name, email, initial password) via GraphQL <code>createUser</code> mutation. Accepts a parameter file or CLI args. Does not assign groups — that is a separate script to allow role assignment to be audited independently.
<code>lldap-user-offboard.sh</code>	Cloud Admin	Disables an LLDAP user account and removes them from all groups. Intended to be called as step 1 of the full offboarding chain (see <code>chain-offboard-full.sh</code>).
<code>lldap-group-add-member.sh</code>	Cloud Admin	Adds an existing user to a named LLDAP group (role). Emits a structured log entry for audit. After completion, triggers the OpenFGA tuple reconciler.
<code>lldap-group-remove-member.sh</code>	Cloud Admin	Removes a user from a named LLDAP group. After completion, triggers <code>openfga-tuple-reconcile.py</code> to delete stale tuples.
<code>lldap-group-create.sh</code>	Cloud Owner	Creates a new LLDAP group following the agreed naming convention (e.g. <code>cloud-admin-aws</code> , <code>cloud-ro-gcp</code>). Idempotent — skips if group already exists.
<code>lldap-group-delete.sh</code>	Cloud Owner	Deletes an LLDAP group after verifying it has zero members. Prevents accidental deletion of populated groups.
<code>lldap-user-password-reset.sh</code>	Cloud Admin	Resets a user's LLDAP password to a generated value and prints it once to stdout for secure hand-off.
<code>lldap-group-list-members.sh</code>	Cloud Admin / Owner	Lists all members of a given LLDAP group. Used for access reviews. Output is JSON for pipeline use or human-readable table.

Script Name	Persona	Purpose
<code>lldap-user-list-groups.sh</code>	Cloud Admin / Owner	Lists all groups a user belongs to. Used during access review or to diagnose unexpected permissions.
<code>lldap-audit-all-memberships.sh</code>	Cloud Owner	Dumps the full LLDAP group→member matrix to a dated CSV. Monthly access review artifact.
<code>lldap-admin-cred-rotate.sh</code>	Cloud Owner	Rotates the LLDAP admin bind password: generates a new secret, writes it to Vault at <code>secret/lldap/admin</code> , updates LLDAP via API, then verifies connectivity.

Group 2 — Authorization Model & Tuple Management (OpenFGA)

These scripts manage relationship tuples and verify authorization state. Model changes themselves are developer work (DSL authoring + CI pipeline), but tuple operations are admin-executable.

Script Name	Persona	Purpose
<code>openfga-tuple-write.sh</code>	Cloud Admin	Writes a single relationship tuple to the OpenFGA store. Usage: <code>openfga-tuple-write.sh user:alice member role:cloud-admin</code> . Used for one-off manual grants that fall outside the reconciler.
<code>openfga-tuple-delete.sh</code>	Cloud Admin	Deletes a specific tuple. Used during manual offboarding or when correcting a misassigned permission.
<code>openfga-tuple-reconcile.py</code>	Cloud Admin (scheduled / triggered)	Reads current LLDAP group memberships via LDAP query and compares them to all tuples in the OpenFGA store. Writes missing tuples and deletes stale ones. Core consistency engine between LLDAP and OpenFGA. Should run on a cron (every 5 minutes) and be callable on demand after any LLDAP group change.
<code>openfga-check.sh</code>	Cloud Admin / Owner	Performs a single Check call: <code>openfga-check.sh user:<id> <relation> <object></code> . Returns allowed/denied. Used to verify permissions before or after a change, and to diagnose access denials.
<code>openfga-list-objects.sh</code>	Cloud Admin	Lists all objects of a given type that a specific user has a given relation to. Example: all <code>cloud:provider</code> resources that <code>user:alice</code> has <code>provision</code> access to.
<code>openfga-list-users.sh</code>	Cloud Admin / Owner	Lists all users who have a given relation to a given object. Used for access reviews: "who can provision on AWS?"
<code>openfga-tuple-audit.py</code>	Cloud Owner	Dumps all tuples in the OpenFGA store and cross-references them against current LLDAP memberships. Flags orphan tuples (user no longer in LLDAP) and missing tuples (LLDAP member has no tuple). Output: dated CSV report.
<code>openfga-breakglass-grant.sh</code>	Cloud Admin	Writes a time-bounded elevated-access tuple and schedules its deletion via a background job or cron entry. Parameters: user, relation, object, TTL in minutes.
<code>openfga-presharedkey-rotate.sh</code>	Cloud Owner	Generates a new preshared key, writes it to Vault at <code>secret/openfga/apikey</code> , updates the OpenFGA server config (or Helm values secret), triggers a rolling restart of the OpenFGA deployment, then updates the libcloud REST configuration to use the new key. Verifies connectivity before exiting.

Script Name	Persona	Purpose
openfga-denial-log-query.sh	Cloud Admin	Queries the structured logs of libcloud REST (or OpenFGA) for all authorization denial events in the last N hours. Formats output by user, resource, and relation for triage.

Group 3 — Vault Operations

Scripts cover policy management, credential operations, lease management, and Vault health. Admins interact with Vault directly only for operational tasks; provisioning credential fetches are internal to libcloud REST.

3.1 — Cloud Owner: Policy & Engine Administration

Script Name	Persona	Purpose
vault-policy-apply.sh	Cloud Owner	Applies a Vault HCL policy file by name: <code>vault-policy-apply.sh cloud-admin-aws policy-files/cloud-admin-aws.hcl</code> . Idempotent. Intended to be called from a GitOps pipeline but also runnable ad hoc.
vault-ldap-group-bind.sh	Cloud Owner	Binds a Vault policy to an LLDAP group via the Vault LDAP auth method: <code>vault write auth/ldap/groups/<group> policies=<policy></code> . Should be run whenever a new LLDAP group is created.
vault-secrets-engine-enable.sh	Cloud Owner	Enables and configures a cloud provider secrets engine (AWS, Azure, GCP, Alibaba) at a given mount path. Reads root credentials from a local env file (never hardcoded). One-off per provider.
vault-role-create.sh	Cloud Owner	Creates a Vault dynamic secret role for a cloud provider: <code>vault-role-create.sh aws ec2-admin '{"Version":"2012-10-17",...}'</code> . The IAM/IAM-equivalent policy document is passed as a file argument.
vault-root-cred-rotate.sh	Cloud Owner	Rotates the root credentials of a cloud secrets engine: <code>vault write <mount>/config/rotate-root</code> . Should be run quarterly per provider. Warning: after rotation Vault holds the only copy of the new root credential.
vault-policy-list-audit.sh	Cloud Owner	Lists all policies in Vault and, for each, lists the LDAP groups and AppRoles bound to it. Produces a dated CSV for compliance review.

3.2 — Cloud Admin: Credential & Lease Operations

Script Name	Persona	Purpose
vault-lease-list.sh	Cloud Admin	Lists all active leases under a given path prefix: <code>vault-lease-list.sh aws/creds/ec2-admin</code> . Useful for understanding what credentials are currently live.
vault-lease-renew.sh	Cloud Admin	Renews a specific lease by ID. Used when automated renewal (Vault Agent) has failed and a service is about to lose access.
vault-lease-revoke.sh	Cloud Admin	Immediately revokes a specific lease. Used during incident response to invalidate a credential that has been exposed.

Script Name	Persona	Purpose
<code>vault-lease-revoke-prefix.sh</code>	Cloud Owner	Revokes all leases under a path prefix (e.g. all AWS credentials for a departing team). High-impact — requires Owner persona.
<code>vault-dynamic-cred-request.sh</code>	Cloud Admin	Manually requests a dynamic credential from Vault for a one-off operation: <code>vault-dynamic-cred-request.sh aws ec2-admin</code> . Credential is printed once to stdout and is time-limited.
<code>vault-static-secret-rotate.sh</code>	Cloud Admin	Rotates a static secret at a given Vault path (e.g. a SaaS API key): generates a new value, writes it to Vault, and optionally calls a provider API to register the new key before deleting the old one.
<code>vault-health-check.sh</code>	Cloud Admin	Calls <code>vault status</code> and key health endpoints; checks seal status, HA standby state, and replication status if applicable. Used in monitoring and as a pre-flight before provisioning operations.
<code>vault-audit-log-query.sh</code>	Cloud Admin / Owner	Queries the Vault audit log (file or syslog sink) for accesses to a given path in the last N hours. Used to verify expected access patterns or investigate anomalous activity.
<code>vault-token-lookup.sh</code>	Cloud Admin	Looks up a Vault token's policies, TTL, and accessor. Used to verify that a service or user token has the expected permissions.

Group 4 — Cloud Provisioning (libcloud REST)

These scripts are thin wrappers around the libcloud REST API. They handle authentication (presenting the user's identity/token so OpenFGA can check the request), provider routing, and sensible defaults. The underlying API calls are cloud-provider-agnostic by design.

4.1 — Compute

Script Name	Persona	Purpose
<code>cloud-node-list.sh</code>	Cloud Admin	Lists all compute nodes for a given provider and region: <code>cloud-node-list.sh aws ap-southeast-1</code> . Returns node ID, name, state, size, and public IP.
<code>cloud-node-provision.sh</code>	Cloud Admin	Provisions a new compute instance using a parameter file: <code>cloud-node-provision.sh --provider aws --params node-spec.json</code> . The spec file defines size, image, region, SSH key, and tags. Idempotent by name.
<code>cloud-node-action.sh</code>	Cloud Admin	Executes a lifecycle action on a node: <code>cloud-node-action.sh <node-id> <start stop reboot destroy></code> . Requires explicit <code>--confirm</code> flag for <code>destroy</code> .
<code>cloud-keypair-manage.sh</code>	Cloud Admin	Creates, lists, or deletes SSH key pairs on a provider: <code>cloud-keypair-manage.sh aws create my-key ~/.ssh/id_rsa.pub</code> .
<code>cloud-image-list.sh</code>	Cloud Admin	Lists available OS images (AMIs, GCP images, etc.) for a provider, optionally filtered by OS family or architecture.
<code>cloud-size-list.sh</code>	Cloud Admin	Lists available instance sizes/flavours for a provider and region, with CPU, RAM, and pricing metadata.

4.2 — Storage

Script Name	Persona	Purpose
cloud-storage-bucket-create.sh	Cloud Admin	Creates an object storage bucket/container on a provider with enforced naming convention and mandatory tags.
cloud-storage-bucket-delete.sh	Cloud Admin	Deletes an empty bucket. Refuses if the bucket contains objects (safety guard).
cloud-storage-bucket-list.sh	Cloud Admin	Lists all buckets for a provider with creation date, region, and object count.
cloud-storage-object-upload.sh	Cloud Admin	Uploads a file to a specified bucket and key path.
cloud-storage-object-download.sh	Cloud Admin	Downloads an object from a bucket to a local path.
cloud-volume-list.sh	Cloud Admin	Lists block storage volumes for a provider and region with state (attached/detached) and size.

4.3 — Networking

Script Name	Persona	Purpose
cloud-network-list.sh	Cloud Admin	Lists VPCs/networks and their subnets for a provider and region.
cloud-floatingip-allocate.sh	Cloud Admin	Allocates a new floating/elastic IP on a provider.
cloud-floatingip-release.sh	Cloud Admin	Releases a floating IP back to the pool.
cloud-floatingip-list.sh	Cloud Admin	Lists all allocated floating IPs and their current association.

Group 5 — Cross-System Chained Operations

These scripts orchestrate actions across multiple systems in a single atomic (or compensating) workflow. They are the most critical operational scripts as they maintain system consistency.

Script Name	Persona	Purpose
chain-onboard-user.sh	Cloud Admin	Full onboarding chain: (1) create LLDAP user, (2) assign to specified groups, (3) trigger <code>openfga-tuple-reconcile.py</code> , (4) verify Vault LDAP auth recognises the new group membership. Accepts a YAML/JSON spec file with user details and role list.
chain-offboard-user.sh	Cloud Admin / Owner	Full offboarding chain: (1) remove user from all LLDAP groups, (2) disable LLDAP account, (3) delete all OpenFGA tuples for the user, (4) revoke all Vault tokens and leases associated with the user, (5) emit a signed offboarding audit record. Must complete all steps or roll back with error report.

Script Name	Persona	Purpose
chain-role-assign.sh	Cloud Admin	Assigns a user to a new role: (1) <code>lldap-group-add-member</code> , (2) <code>openfga-tuple-write</code> for the new role relation, (3) verify <code>openfga-check</code> returns allowed for a representative resource.
chain-role-revoke.sh	Cloud Admin	Revokes a role from a user: (1) <code>lldap-group-remove-member</code> , (2) <code>openfga-tuple-delete</code> , (3) <code>vault-lease-revoke-prefix</code> for any leases tied to that role path, (4) verify <code>openfga-check</code> returns denied.
chain-provider-onboard.sh	Cloud Owner	Pre-work for a new cloud provider: (1) enable Vault secrets engine, (2) create Vault roles per permission level, (3) create LLDAP groups for the provider, (4) bind Vault policies to groups, (5) validate end-to-end with a test credential request. (Developer must have already added the libcloud driver and OpenFGA relations.)
chain-diagnose-access.sh	Cloud Admin	Diagnostic chain for "user X cannot do Y on Z": (1) checks LLDAP group membership, (2) checks OpenFGA tuple existence, (3) calls <code>openfga-check</code> , (4) checks Vault policy for the user's role, (5) prints a structured triage report indicating which layer is denying access.
chain-presharedkey-rotate.sh	Cloud Owner	Rotates the OpenFGA preshared key end-to-end: writes new key to Vault, updates OpenFGA server config, performs rolling restart, updates libcloud REST env/config, and smoke-tests an authorization check to confirm the new key is live.

Group 6 — Reconciliation & Health (Scheduled)

These scripts are intended to run on a cron schedule and alert on drift. Cloud Admins monitor their output; Cloud Owners govern the schedule.

Script Name	Schedule	Persona (monitors)	Purpose
openfga-tuple-reconcile.py	Every 5 min	Cloud Admin	LLDAP → OpenFGA sync (see Group 2). Primary consistency enforcer.
reconcile-vault-ldap-bindings.sh	Every 15 min	Cloud Owner	Compares <code>auth/ldap/groups</code> entries in Vault to current LLDAP groups. Alerts if a group exists in LLDAP but has no Vault binding, or if a Vault binding references a deleted LLDAP group.
audit-orphan-tuples.py	Daily	Cloud Owner	Identifies OpenFGA tuples where the user no longer exists in LLDAP. Outputs a report; does not auto-delete (requires human confirmation).
vault-lease-expiry-monitor.sh	Every 10 min	Cloud Admin	Lists all leases expiring within the next N minutes across all cloud secrets engine paths. Triggers renewal for renewable leases; pages on-call for non-renewable ones.
cloud-node-state-report.sh	Daily	Cloud Admin	Generates a cross-provider report of all running nodes, their age, and their assigned tags. Flags nodes older than a configurable threshold for review.
lldap-audit-all-memberships.sh	Monthly	Cloud Owner	Full group membership dump (see Group 1). Sent to access review workflow.
openfga-tuple-audit.py	Monthly	Cloud Owner	Full tuple vs LLDAP diff (see Group 2). Compliance artifact.

Script Name	Schedule	Persona (monitors)	Purpose
vault-policy-list-audit.sh	Monthly	Cloud Owner	Full Vault policy → group binding audit (see Group 3). Compliance artifact.

Summary: Script Count by Layer and Persona

Layer	Cloud Owner Scripts	Cloud Admin Scripts	Shared
LLDAP	4	6	1
OpenFGA	3	5	2
Vault (policy/engine)	6	0	0
Vault (credential/lease)	2	6	2
libcloud REST (compute)	0	6	0
libcloud REST (storage)	0	6	0
libcloud REST (network)	0	4	0
Cross-system chains	3	4	0
Reconciliation / health	3	3	2
Total	21	40	7